

Lab-02

Task 1-A: [1 3 5 7]

↓
start

Here, I just use nested loops where I compared 3 element with every item at the loop. If both items sum is equal to my desired number then just stored the index number. The code was giving time complexity as $O(n^3)$.

Task 1-B: [1 5 8 9 9 10]

i →

← j

Here I just used positive and negative indexing. One variable started from first index and one from last index. If both's sum equals to desired number then just return. If we do not get our desired sum then just return "Impossible".

Task-2a:

Here ~~two~~ two lists are given. I simply merged them and then used sort function.

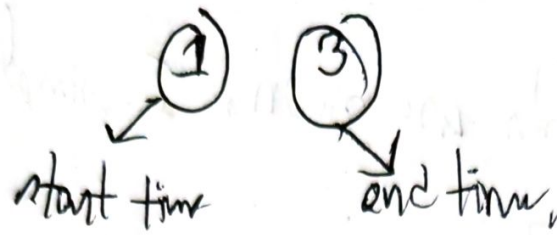
It's giving the time complexity as $O(n \log n)$

Task-2b:

Here I used concept as merge sort. Took a new list ~~and~~ Then used forward indexing in both lists. ~~Both~~

If one item is greater than other then I put the smaller one in new list. By comparing every item started whole list. If one list item ends then add the rest of the array in the list.

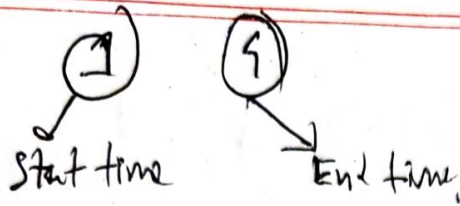
Task-3



If a worker wants to do a task at first he has to finish then if it ~~clashes~~ does not clash with another start time he can do the task.

At first took a new array and inserted all the times. ~~and~~ As all our tasks depend on end time. So sorted array on the basis of end time. Then check how much ~~more~~ time that end time and next worker's start time doesn't clash.

Task-4



At task 3 it was for single person but this time it's for multiple person.

First part is similar to task-3. Here I used merge sort to sort the times. I took an array having next available time and a index to compare. First gave the first task to first person. Then compared if second person is available that time if he is available the index changes ~~to~~ and count increases. At last ~~is~~ after reviewing all items printed the count.